

View Only

QUESTIONS

Q1
AI-Based Student Performance
Analytics System
10 marks

Q2
Digital Healthcare Patient
Monitoring System
10 marks

Q3
Digital Wallet and FinTech
Transaction System
10 marks

Q4
Smart Agriculture Crop
Monitoring System
10 marks

Q5
Smart City IoT Energy
Management System
10 marks

5/5 answered

Q1 AI-Based Student Performance Analytics System

10 marks

AI-Based Student Performance Analytics System ♦ Scenario: Intelligent Education. A university wants to develop an AI-assisted Student Performance Analytics System to monitor student academic performance and identify students who may need additional academic support.

- Create a class Student with attributes such as Student ID, Name, Marks in different subjects, Average, and Performance Level.
- Implement member functions to input student details, calculate average marks, determine performance level, and display the student report.
- Use an array of objects to manage records of multiple students.
- Include a static member to maintain the total number of students analyzed.
- Apply proper encapsulation using private data members and public member functions.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Student {
6 private:
7     int id;
8     string name;
9     float marks[3];
10    float average;
11    string level;
12
13    static int totalStudents;
14
15 public:
16    void input() {
17        cout << "Enter Student ID: ";
18        cin >> id;
19
20        cout << "Enter Name: ";
21        cin >> name;
22
23        cout << "Enter marks in 3 subjects: ";
24        for (int i = 0; i < 3; i++)
25            cin >> marks[i];
26
27        totalStudents++;
28    }
29
30    void calculate() {
31        average = (marks[0] + marks[1] + marks[2]) / 3;
32
33        if (average >= 80)
34            level = "Excellent";
35        else if (average >= 60)
36            level = "Good";
37        else if (average >= 40)
38            level = "Average";
39        else
40            level = "Needs Improvement";
41    }
42
43    void display() {
44        cout << "\nStudent ID: " << id;
45        cout << "\nName: " << name;
46        cout << "\nAverage: " << average;
47        cout << "\nPerformance Level: " << level << endl;
48    }
49
50    static void showTotal() {
51        cout << "\nTotal Students Analyzed: " << totalStudents << endl;
52    }
53 };
54
55 int Student::totalStudents = 0;
56
57 int main() {
58     int n;
59
60     cout << "Enter number of students: ";
61     cin >> n;
62
63     Student s[50];
64
65     for (int i = 0; i < n; i++) {
66         cout << "\nEnter details of Student " << i + 1 << endl;
67         s[i].input();
68         s[i].calculate();
69     }
70
71     cout << "\n--- Student Performance Report ---\n";
72
73     for (int i = 0; i < n; i++)
74         s[i].display();
75
76     Student::showTotal();
77
78     return 0;
79 }
```

Input

2

1

101

Ravi

Output

QUESTIONS

Q1

AI-Based Student Performance
Analytics System
10 marks

Q2

Digital Healthcare Patient
Monitoring System
10 marks

Q3

Digital Wallet and FinTech
Transaction System
10 marks

Q4

Smart Agriculture Crop
Monitoring System
10 marks

Q5

Smart City IoT Energy
Management System
10 marks

5/5 answered

Q2 Digital Healthcare Patient Monitoring System

10 marks

Digital Healthcare Patient Monitoring System ♦ Scenario: Smart Healthcare. A modern hospital wants to implement a Digital Patient Monitoring System that maintains patient information and automatically calculates basic health-risk indicators from recorded medical parameters.

- Design a class Patient with attributes such as Patient ID, Name, Age, Heart Rate, Blood Pressure, and Health Status.
- Implement member functions to input patient details, calculate/identify the health status based on predefined conditions, and display the patient report.
- Store multiple patient records using an array of objects.
- Use a static member to track the total number of patients monitored.
- Demonstrate data hiding, encapsulation, and structured object-oriented program design.

Your Code

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Patient {
6 private:
7     int id, age, heartRate, bp;
8     string name, healthStatus;
9
10    static int totalPatients;
11
12 public:
13    void input() {
14        cout << "Enter Patient ID: ";
15        cin >> id;
16
17        cout << "Enter Name: ";
18        cin >> name;
19
20        cout << "Enter Age: ";
21        cin >> age;
22
23        cout << "Enter Heart Rate: ";
24        cin >> heartRate;
25
26        cout << "Enter Blood Pressure: ";
27        cin >> bp;
28
29        totalPatients++;
30    }
31
32    void checkHealth() {
33        if (heartRate >= 60 && heartRate <= 100 &&
34            bp >= 90 && bp <= 120)
35            healthStatus = "Normal";
36        else
37            healthStatus = "Needs Attention";
38    }
39
40    void display() {
41        cout << "\nPatient ID: " << id;
42        cout << "\nName: " << name;
43        cout << "\nAge: " << age;
44        cout << "\nHeart Rate: " << heartRate;
45        cout << "\nBlood Pressure: " << bp;
46        cout << "\nHealth Status: " << healthStatus << endl;
47    }
48
49    static void showTotal() {
50        cout << "\nTotal Patients Monitored: " << totalPatients << endl;
51    }
52 };
53
54 int Patient::totalPatients = 0;
55
56 int main() {
57     int n;
58
59     cout << "Enter number of patients: ";
60     cin >> n;
61
62     Patient p[50];
63
64     for (int i = 0; i < n; i++) {
65         cout << "\nEnter details of Patient " << i + 1 << endl;
66         p[i].input();
67         p[i].checkHealth();
68     }
69
70     cout << "\n--- Patient Health Report ---\n";
71
72     for (int i = 0; i < n; i++)
73         p[i].display();
74
75     Patient::showTotal();
76
77     return 0;
78 }

```

Input

```

2
1

```

View Only

QUESTIONS

Q1 ✓
AI-Based Student Performance Analytics System
10 marks

Q2 ✓
Digital Healthcare Patient Monitoring System
10 marks

Q3 ✓
Digital Wallet and FinTech Transaction System
10 marks

Q4 ✓
Smart Agriculture Crop Monitoring System
10 marks

Q5 ✓
Smart City IoT Energy Management System
10 marks

5/5 answered

Q3 Digital Wallet and FinTech Transaction System

10 marks

Digital Wallet and FinTech Transaction System ♦ Scenario: Cashless Economy. A FinTech company wants to develop a Digital Wallet Management System to maintain customer wallet information and perform secure financial transactions.

- Create a class DigitalWallet with attributes such as Wallet ID, Customer Name, Mobile Number, and Balance.
- Implement member functions for wallet creation, money deposit, money withdrawal, balance enquiry, and transaction display.
- Use an array of objects to manage multiple customer wallets.
- Include a static member to maintain the total number of active wallets.
- Apply appropriate access specifiers and encapsulation to prevent direct modification of sensitive financial data.

Your Code

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 class DigitalWallet {
7 private:
8     int walletID;
9     string customerName;
10    string mobileNumber;
11    double balance;
12    static int activeWallets;
13
14 public:
15    DigitalWallet() {
16        walletID = 0;
17        customerName = "";
18        mobileNumber = "";
19        balance = 0.0;
20    }
21
22    void createWallet(int id, string name, string mobile, double initialBalance) {
23        walletID = id;
24        customerName = name;
25        mobileNumber = mobile;
26        balance = initialBalance;
27        activeWallets++;
28    }
29
30    void deposit(double amount) {
31        if (amount > 0) {
32            balance += amount;
33            cout << "Deposited: " << amount << " | Balance: " << balance << endl;
34        }
35    }
36
37    void withdraw(double amount) {
38        if (amount > 0 && amount <= balance) {
39            balance -= amount;
40            cout << "Withdrew: " << amount << " | Balance: " << balance << endl;
41        } else {
42            cout << "Insufficient balance or invalid amount." << endl;
43        }
44    }
45
46    void balanceEnquiry() const {
47        cout << "Wallet ID " << walletID << " Balance: " << balance << endl;
48    }
49
50    void displayDetails() const {
51        cout << "ID: " << walletID << ", Name: " << customerName << ", Mobile: " << mobileNumber << endl;
52    }
53
54    static int getActiveWallets() {
55        return activeWallets;
56    }
57 };
58
59 int DigitalWallet::activeWallets = 0;
60
61 int main() {
62    DigitalWallet wallets[2];
63
64    wallets[0].createWallet(101, "Alice", "9876543210", 500.0);
65    wallets[1].createWallet(102, "Bob", "9123456789", 1000.0);
66
67    wallets[0].displayDetails();
68    wallets[0].deposit(200.0);
69    wallets[0].withdraw(100.0);
70    wallets[0].balanceEnquiry();
71
72    cout << "---" << endl;
73
74    wallets[1].displayDetails();
75    wallets[1].withdraw(1200.0);
76
77    cout << "---" << endl;
78    cout << "Total Active Wallets: " << DigitalWallet::getActiveWallets() << endl;
79
80    return 0;
81 }
```

Input

```
2
101 Alice 99976345612 500
102 Bob 1234567894 1000
101
```

Output

QUESTIONS

Q1

AI-Based Student Performance
Analytics System
10 marks

Q2

Digital Healthcare Patient
Monitoring System
10 marks

Q3

Digital Wallet and FinTech
Transaction System
10 marks

Q4

Smart Agriculture Crop
Monitoring System
10 marks

Q5

Smart City IoT Energy
Management System
10 marks

5/5 answered

Q4 Smart Agriculture Crop Monitoring System

10 marks

Smart Agriculture Crop Monitoring System ♦ Scenario: IoT-Based Agriculture. An agricultural research centre is developing an IoT-based Smart Agriculture System to monitor crop fields using sensor data and identify whether crops require irrigation.

- Design a class CropField with attributes such as Field ID, Crop Name, Soil Moisture, Temperature, Humidity, and Irrigation Status.
- Implement member functions to input sensor readings, evaluate soil conditions, determine irrigation requirements, and display field information.
- Use an array of objects to maintain information about multiple crop fields.
- Include a static member to count the total number of monitored fields.
- Ensure proper encapsulation and modular object-oriented design suitable for an IoT-based agricultural application.

Your Code

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class CropField {
6 private:
7     int fieldID;
8     string cropName;
9     float soilMoisture;
10    float temperature;
11    float humidity;
12    bool irrigationStatus;
13
14    static int totalFields;
15
16 public:
17     CropField() {
18         fieldID = 0;
19         cropName = "";
20         soilMoisture = 0.0;
21         temperature = 0.0;
22         humidity = 0.0;
23         irrigationStatus = false;
24         totalFields++;
25     }
26
27     void inputData() {
28         cin >> fieldID >> cropName >> soilMoisture >> temperature >> humidity;
29         evaluateIrrigation();
30     }
31
32     void evaluateIrrigation() {
33         if (soilMoisture < 30.0) {
34             irrigationStatus = true;
35         } else {
36             irrigationStatus = false;
37         }
38     }
39
40     void displayInfo() const {
41         cout << "\n--- Field Information ---" << endl;
42         cout << "Field ID: " << fieldID << endl;
43         cout << "Crop Name: " << cropName << endl;
44         cout << "Soil Moisture: " << soilMoisture << "%" << endl;
45         cout << "Temperature: " << temperature << "°C" << endl;
46         cout << "Humidity: " << humidity << "%" << endl;
47         cout << "Irrigation Status: "
48             << (irrigationStatus ? "Required (Soil Moisture Low)" : "Not Required")
49             << endl;
50     }
51
52     static int getTotalFields() {
53         return totalFields;
54     }
55 };
56
57 int CropField::totalFields = 0;
58
59 int main() {
60     int numFields;
61     if (!(cin >> numFields) || numFields <= 0) {
62         return 0;
63     }
64
65     CropField* fields = new CropField[numFields];
66
67     for (int i = 0; i < numFields; i++) {
68         fields[i].inputData();
69     }
70
71     cout << "Total Monitored Fields: " << CropField::getTotalFields() << endl;
72
73     for (int i = 0; i < numFields; i++) {
74         fields[i].displayInfo();
75     }
76
77     delete[] fields;
78     return 0;
79 }

```

View Only

QUESTIONS

Q1 ✓
AI-Based Student Performance
Analytics System
10 marks

Q2 ✓
Digital Healthcare Patient
Monitoring System
10 marks

Q3 ✓
Digital Wallet and FinTech
Transaction System
10 marks

Q4 ✓
Smart Agriculture Crop
Monitoring System
10 marks

Q5 ✓
Smart City IoT Energy
Management System
10 marks

5/5 answered

Q5 Smart City IoT Energy Management System

10 marks

Smart City IoT Energy Management System ♦ Scenario: Sustainable Smart City. A smart-city research project aims to develop an IoT-based Energy Management System for monitoring electricity consumption in different buildings and identifying buildings with high energy usage.

- Create a class Building with attributes such as Building ID, Building Name, Building Type, Energy Consumption, and Energy Status.
- Implement member functions to input building details, calculate energy status based on consumption, and display energy usage information.
- Use an array of objects to manage multiple buildings in the smart-city system.
- Include a static member to maintain the total number of buildings monitored.
- Apply encapsulation, data hiding, and structured C++ object-oriented programming principles in the implementation.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Building {
6 private:
7     int buildingID;
8     string buildingName;
9     string buildingType;
10    float energyConsumption;
11    string energyStatus;
12
13    static int totalBuildings;
14
15 public:
16     Building() {
17         buildingID = 0;
18         buildingName = "";
19         buildingType = "";
20         energyConsumption = 0.0;
21         energyStatus = "Normal";
22         totalBuildings++;
23     }
24
25     void inputDetails() {
26         cin >> buildingID >> buildingName >> buildingType >> energyConsumption;
27         calculateEnergyStatus();
28     }
29
30     void calculateEnergyStatus() {
31         if (energyConsumption > 1000.0) {
32             energyStatus = "High Consumption";
33         } else if (energyConsumption >= 500.0) {
34             energyStatus = "Moderate Consumption";
35         } else {
36             energyStatus = "Low Consumption";
37         }
38     }
39
40     void displayDetails() const {
41         cout << "\n--- Building Energy Details ---" << endl;
42         cout << "Building ID: " << buildingID << endl;
43         cout << "Building Name: " << buildingName << endl;
44         cout << "Building Type: " << buildingType << endl;
45         cout << "Energy Consumption: " << energyConsumption << " kWh" << endl;
46         cout << "Energy Status: " << energyStatus << endl;
47     }
48
49     static int getTotalBuildings() {
50         return totalBuildings;
51     }
52 };
53
54 int Building::totalBuildings = 0;
55
56 int main() {
57     int n;
58     if (!(cin >> n) || n <= 0) {
59         return 0;
60     }
61
62     Building* buildings = new Building[n];
63
64     for (int i = 0; i < n; i++) {
65         buildings[i].inputDetails();
66     }
67
68     cout << "Total Monitored Buildings: " << Building::getTotalBuildings() << endl;
69
70     for (int i = 0; i < n; i++) {
71         buildings[i].displayDetails();
72     }
73
74     delete[] buildings;
75     return 0;
76 }
```

Input

```
2
201 Techpark Commercial 1500
202 Greenview Residential 350
```

Output